

Group number: 65

Module: 6

Emrik Dunvald 020208-5759, ADS

Elias Samantzis 000715-6631, ADS

Gusamanel@student.gu.se

gusdunvem@student.gu.se

We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part of other solutions.

Elias hours spent: 20 hours

Emrik hours spent: 20 hours

Paper summary

This paper discussed the model AlphaGo which combines traditional monte carlo rollout strategies with neural networks in order to more intelligently select moves. Go is a challenging game for search trees due to the number of possible moves a player could choose at any given time. This model limits the search space using convolutional neural networks which are trained on expert level games to predict what kinds of decisions a professional would make at different board positions. This, combined with value networks and monte carlo tree search, allows the model to surpass other models which rely only on simulations. The model also trains a neural network to predict values of different board positions in order to evaluate different moves. This is then further improved by having the model play games against previous iterations of itself as a way of reinforcement learning where winning the game rewards it.

There are many challenges associated with training such a complex architecture such as overfitting which is addressed in several ways. Another issue is computational resources which the model demands due to needing to perform both simulations, compute policy networks and compute value networks. The results are however remarkable as presented in the paper where AlphaGo beat all other existing programs most if not all the time. It also beat the best human player at the time which was a feat believed to be impossible.

```
import random
import math
import time
import pandas as pd

class Node:
    def __init__(self, state, parent=None):
        self.state = state
        self.parent = parent
```

```

        self.children = []
        self.visits = 0
        self.value = 0

    def is_fully_expanded(self):
        # Fully expanded if it has as many children as legal moves
        return len(self.children) == len(self.state.get_legal_moves())

    def best_child(self, exploration_weight=1.4):
        # Selecting the best child based on the UCT formula
        return max(self.children, key=lambda node: node.value /
(node.visits + 1e-6) +
                    exploration_weight * math.sqrt(math.log(self.visits
+ 1) / (node.visits + 1e-6)))

# Monte Carlo Tree Search algorithm, where we iteratively select and
expand nodes to find the best move
def monte_carlo_tree_search(root, iterations=1000):
    for _ in range(iterations):
        node = root
        while node.children:
            node = node.best_child()
            if not node.state.is_terminal() and not
node.is_fully_expanded():
                node = expand(node)
            result = simulate(node)
            backpropagate(node, result)
        return root.best_child(exploration_weight= 1.4)

# Expanding a node by adding a child for an unexplored move
def expand(node):
    legal_moves = node.state.get_legal_moves()
    unexplored_moves = [move for move in legal_moves if move not in
[child.state for child in node.children]]
    move = random.choice(unexplored_moves)
    new_state = node.state.apply_move(move)
    child_node = Node(new_state, parent=node)
    node.children.append(child_node)
    return child_node

# Simulating a random rollout from the given node to a final state
def simulate(node):
    state = node.state.clone()
    while not state.is_terminal():
        legal_moves = state.get_legal_moves()

        for move in legal_moves:
            new_state = state.apply_move(move)
            if new_state.get_result() == state.current_player:
                return state.current_player

```

```

        move = random.choice(legal_moves)
        state = state.apply_move(move)

    return state.get_result()

# A method to update visit counts and win values up the tree
def backpropagate(node, result):
    while node is not None:
        node.visits += 1
        if result == node.state.current_player:
            node.value += 1
        elif result == 0:
            node.value += 0
        else:
            node.value -= 1
        node = node.parent

# Game implementation
class TicTacToe:
    def __init__(self, size=3):
        self.size = size
        self.board = [0] * (size * size)
        self.current_player = 1

    def get_legal_moves(self):
        return [i for i in range(self.size * self.size) if
self.board[i] == 0]

    def apply_move(self, move):
        new_state = TicTacToe(size=self.size)
        new_state.board = self.board[:]
        new_state.board[move] = self.current_player
        new_state.current_player = -self.current_player
        return new_state

    def is_terminal(self):
        return self.get_result() is not None

    def get_result(self):
        size = self.size
        board = self.board

        if size == 3:
            win_length = 3
        elif size == 4:
            win_length = 4
        else:
            win_length = 5

```

```

    # Checking rows
    for i in range(size):
        for j in range(size - win_length + 1):
            if board[i * size + j] != 0 and all(
                board[i * size + j] == board[i * size + j + k] for
k in range(win_length)
            ):
                return board[i * size + j]

    # Checking columns
    for j in range(size):
        for i in range(size - win_length + 1):
            if board[i * size + j] != 0 and all(
                board[i * size + j] == board[(i + k) * size + j]
for k in range(win_length)
            ):
                return board[i * size + j]

    # Checking left diagonal \
    for i in range(size - win_length + 1):
        for j in range(size - win_length + 1):
            if board[i * size + j] != 0 and all(
                board[i * size + j] == board[(i + k) * size + (j +
k)] for k in range(win_length)
            ):
                return board[i * size + j]

    # Checking right diagonal /
    for i in range(size - win_length + 1):
        for j in range(win_length - 1, size):
            if board[i * size + j] != 0 and all(
                board[i * size + j] == board[(i + k) * size + (j -
k)] for k in range(win_length)
            ):
                return board[i * size + j]

    # If no winner and board is full, return 0 meaning draw
    if 0 not in board:
        return 0

    return None

# Creating a copy of the current game state
def clone(self):
    new_state = TicTacToe(size=self.size)
    new_state.board = self.board[:]
    new_state.current_player = self.current_player
    return new_state

```

```

# Running the game

game = TicTacToe()
def print_board(game):
    for i in range(game.size):
        print(game.board[i * game.size:(i + 1) * game.size])
    print()

while not game.is_terminal():
    print_board(game)
    root = Node(game)
    if game.current_player == 1:
        best_node = monte_carlo_tree_search(root, iterations=5000)
        best_move = best_node.state
    else:
        random_move = random.choice(game.get_legal_moves())
        best_move = game.apply_move(random_move)

    game = best_move
    print("Move made!")

print("Final Board:")
for i in range(game.size):
    print(game.board[i * game.size:(i + 1) * game.size])
print("Game Over! Winner:", game.get_result())

[0, 0, 0]
[0, 0, 0]
[0, 0, 0]

Move made!
[0, 0, 0]
[1, 0, 0]
[0, 0, 0]

Move made!
[0, 0, 0]
[1, 0, 0]
[0, -1, 0]

Move made!
[0, 1, 0]
[1, 0, 0]
[0, -1, 0]

Move made!
[0, 1, 0]
[1, 0, -1]
[0, -1, 0]

```

Move made!

[0, 1, 0]

[1, 0, -1]

[1, -1, 0]

Move made!

[-1, 1, 0]

[1, 0, -1]

[1, -1, 0]

Move made!

[-1, 1, 0]

[1, 0, -1]

[1, -1, 1]

Move made!

[-1, 1, -1]

[1, 0, -1]

[1, -1, 1]

Move made!

Final Board:

[-1, 1, -1]

[1, 1, -1]

[1, -1, 1]

Game Over! Winner: 0

Choices and Motivations:

Roll-out policy: Our choice for the roll-out policy follows mostly a random strategy with a small tactical element. This means that when the Monte Carlo Tree Search (MCTS) reaches a new node it will play random moves until the game reaches an end. However, during each move the algorithm will first check if any of the legal moves results in an immediate win for the player, and if such move is detected, then it will play it immediately. This should help the algorithm identify winning tactics, while also simulating diverse game outcomes. Also, with this type of strategy no domain-specific knowledge is needed for the game, meaning that this type of strategy ensures that the algorithm will be able to play the game. This strategy will also ensure robustness to different types of strategies that might be employed by opponents. Since, the algorithm does not assume any specific strategy for the opponent, meaning that this algorithm can go against any opponent and still perform.

Opponent policy: In our implementation the opponent is playing randomly basically, meaning it chooses one of the available moves at random. We implemented this because we wanted to keep the opponent simple and unpredictable, so that we could focus on only testing the performance of Monte carlo tree search. With an opponent employing a random strategy, we aimed to explore a variety of game situations that could occur, since random moves can lead to a variety of different outcomes. In theory this is supposed to help the MTCS learn how to deal with

unexpected and random situations, in order to make it adaptable. Another reason for this choice is to avoid overfitting, as if the MCTS would face a predictable opponent, it would resort to simply learning that specific pattern of a player.

Selection policy: For the selection policy in the search tree, we used the Upper Confidence Bound for Trees (UCT) formula. This balances exploration (trying new or less-visited moves) and exploitation (focusing on moves that have shown good results so far). The UCT formula ensures that the algorithm does not only stick to known good moves but also explores other possibilities, which is important for finding the best overall strategy. This balance ensures that promising moves are chosen more often but does not completely ignore unexplored options.

Back Up policy: For the back-up policy (updates) we decided to use backpropagation to update the values of each node. We update the visit count and the value of each node. Every time a simulation passes through a node we increase its visit count and adjust its value based on whether the final result of the simulation was a loss or a win. This helps the algorithm keep track of how often a move has been executed and how successful it was, and this will allow the algorithm to make more informed decisions over time.

```
import random
import math
import time
import pandas as pd

# Evaluate the Monte Carlo Tree Search performance
def evaluate_mcts(size=3, iterations=1000, games=100):
    total_time = 0
    wins, losses, draws = 0, 0, 0

    for _ in range(games):
        game = TicTacToe(size=size)
        start_time = time.time()

        while not game.is_terminal():
            root = Node(game)
            best_move = monte_carlo_tree_search(root,
iterations=iterations)
            game = best_move.state

        total_time += time.time() - start_time
        result = game.get_result()
        if result == 1:
            wins += 1 # Our algorithm won
        elif result == -1:
            losses += 1 # Our algorithm lost
        else:
            draws += 1 # Draw

    avg_time_per_game = total_time / games
    win_rate = wins / games
    loss_rate = losses / games
```

```

draw_rate = draws / games

return {
    "Grid Size": f"{size}x{size}",
    "Iterations per Move": iterations,
    "Games Played": games,
    "Avg Time per Game (s)": avg_time_per_game,
    "Win Rate": win_rate,
    "Loss Rate": loss_rate,
    "Draw Rate": draw_rate
}

grid_size = 3
mcts_iterations = 5000
num_games = 50

# Run evaluation on the algorithm
mcts_stats = evaluate_mcts(size=grid_size, iterations=mcts_iterations,
games=num_games)

df_stats = pd.DataFrame([mcts_stats])
print(df_stats)

```

	Grid Size	Iterations per Move	Games Played	Avg Time per Game (s)
0	3x3	5000	50	0.713939

	Win Rate	Loss Rate	Draw Rate
0	0.58	0.24	0.18

Performance:

When iterations=1000, games=100 and Grid = 3x3:

	Avg Time per Game (s)	Win Rate	Loss Rate	Draw Rate
0	0.165785	0.52	0.48	0.0

When iterations=1000, games=100 and Grid = 4x4:

	Avg Time per Game (s)	Win Rate	Loss Rate	Draw Rate
0	1.613121	0.09	0.04	0.87

When iterations=1000, games=100 and Grid = 5x5:

	Avg Time per Game (s)	Win Rate	Loss Rate	Draw Rate
0	4.854309	0.06	0.0	0.94

When iterations=5000, games=50 and Grid = 3x3:

	Grid Size	Iterations per Move	Games Played	Avg Time per Game (s)	\
0	3x3	5000	50	0.713939	
	Win Rate	Loss Rate	Draw Rate		
0	0.58	0.24	0.18		

When iterations=5000, games=50 and Grid = 4x4:

	Grid Size	Iterations per Move	Games Played	Avg Time per Game (s)	\
0	4x4	5000	50	8.069008	
	Win Rate	Loss Rate	Draw Rate		
0	0.02	0.0	0.98		

When iterations=50000, games=50 and Grid = 3x3:

	Grid Size	Iterations per Move	Games Played	Avg Time per Game (s)	\
0	3x3	50000	50	10.992837	
	Win Rate	Loss Rate	Draw Rate		
0	0.6	0.4	0.0		

When iterations=50000, games=50 and Grid = 4x4:

	Grid Size	Iterations per Move	Games Played	Avg Time per Game (s)	\
0	4x4	50000	50	76.575672	
	Win Rate	Loss Rate	Draw Rate		
0	0.0	0.0	1.0		

When iterations=50000, games=50 and Grid = 5x5:

	Grid Size	Iterations per Move	Games Played	Avg Time per Game (s)	\
0	4x4	50000	50	74.411704	
	Win Rate	Loss Rate	Draw Rate		
0	0.0	0.0	1.0		

Pros and cons

Pros:

MCTS runs fairly fast, considering the number of iterations and games played each simulation. The games range from 7 seconds to some minutes as the games and iterations increase. However, the win rate seems to fluctuate as the iterations and number of games change, but on average we have a win rate ranging from 50-60%. This means the algorithm can make decisions quickly however, this is the case when the grid is fairly simple (3x3). Even when the grid is larger, the games run relatively fast, considering that they are fully simulated and that there are so many possible scenarios. Even then, we are simulating between 50-100 games with 1000-50000 iterations, basically meaning that given the time it takes to finish running all these games, it is relatively fast, in our opinion. The algorithm is also flexible because it explores many different move sequences rather than following a fixed strategy and this in theory should help it adapt to different opponents' play styles. The algorithm seems to manage to keep a higher win rate than loss rate as the grid size increases, even though, the win rate is extremely low at around 0.7% in those cases. This indicates that the algorithm is somewhat competitive against an opponent that employs the a complete random roll-out strategy. So, in the end our implementation seems to at least be able to keep it self from losing in the majority of cases where we tested it out.

Another advantage is that MCTS uses Upper Confidence Bounds for Trees (UCT), which balances exploration and exploitation. This means it does not only keep choosing the best-known move but also tries out new options to improve its understanding of the game.

Cons:

Considering the results when increasing the grids, our system does not scale well as the grids grow larger. As we can see above in the screenshots, the larger the grids, the smaller the win rate and also we can see that the game time increases substantially. This indicates that our system struggles to make the correct moves that would lead to victory. It becomes evident, that the larger the grids the more computationally heavy the algorithm becomes and the results consistently return draws. This is a result of the exponential increase in possible legal moves, that can be taken by the algorithms. As the grid sizes grow so does the possible moves. This makes it harder for the algorithm to simulate all possibilities efficiently, which is one of the reasons we get draws. The properties of the random roll-outs seems to be a disadvantage here, as it becomes more difficult for the algorithms to find winning sequences by randomly exploring different paths. As the grid sizes increase, the chances of stumbling upon these victory paths diminish, and as a result, the algorithms struggle to find good moves and instead end up with draws.

The algorithm's competitiveness diminishes as the grid size grows. They become less and less confident in their ability to produce victories and resort to more defensive strategies that ensure that, at the very least, a neutral outcome (draw) is produced. This is due to the computational limitations of following the 'random strategy' that we employ. The more possible legal moves there are, the harder it becomes to randomly choose the correct path. However, when the grid size is 3x3, the algorithm remains quite competitive as it manages to achieve a win rate ranging from 50-60% depending on the grid size and iterations and games, in most experiments we ran.

Based on the results from running this Monte Carlo Tree Search algorithm, it does not seem very sample efficient. For example, in the 3x3 grid, the algorithm needed 1000 iterations per move to get a win rate of around 52%. When the number of simulations increased to 50000, the win rate only improved to 60%, which is not a big improvement for so many extra simulations. This shows that the algorithm does not get much better even when it is given a lot more chances to learn.

On larger boards, like 4x4 and 5x5, the algorithms performance was even worse. Even after 50000 simulations per move it could almost never win a game and most games ended in a draw. This means that the algorithm used a huge number of simulations but still could not find good strategies to win. The larger the board gets, the harder it is for the algorithm to figure out what to do even with thousands of simulations, leading us to conclude that the sample is not efficient.

Would you as a human beat it?

Our short answer is yes, we think we would have a good chance to beat this algorithm, especially on the 4x4 and 5x5 grids. The algorithm struggles a lot on larger boards, where most games end in a draw and it almost never wins. This tells us that it is not very good at finding strategies to win and instead it just plays safe moves that avoid losing. However, on the 3x3 grid, it would probably be a bit harder to beat, since it has more experience and manages to win at most around 60% of the games. Although, it required a lot of iteration to achieve that result. We believe with some training that we could confidently beat it even on a 3x3 grid without any concerns. Overall, we believe that we could beat this implementation.

